

Bootstrap Capability

Prompt-Managed Workflow Engineering

NOAA EMC Global Workflow MCP Team

Environmental Modeling Center
National Oceanic and Atmospheric Administration

January 6, 2026

Agenda

- 1 The Problem
- 2 The Solution
- 3 Execution Model
- 4 Side-Effect Classification
- 5 The Iterative Loop
- 6 Workflow Format
- 7 Bootstrap Demo
- 8 Future Work

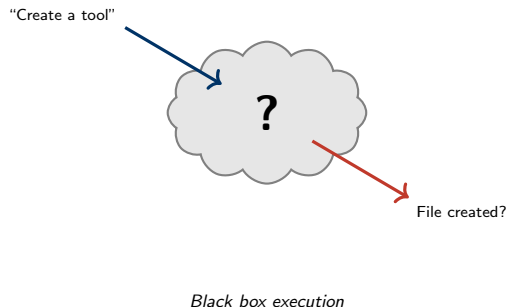
The Problem: AI Code Generation Gap

Current Challenges

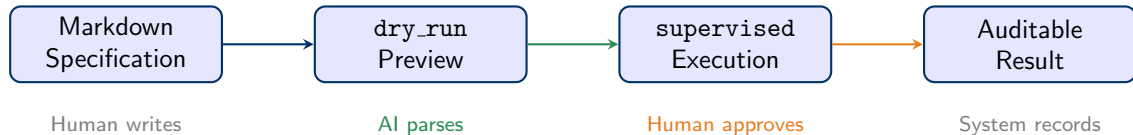
- AI generates code without **execution plan visibility**
- No preview of what will change before it changes
- Difficult to audit or roll back autonomous actions
- Human oversight requires manual intervention

The Gap

Natural language intent → ??? → Executable code



The Solution: Spec-Driven Development (SDD)



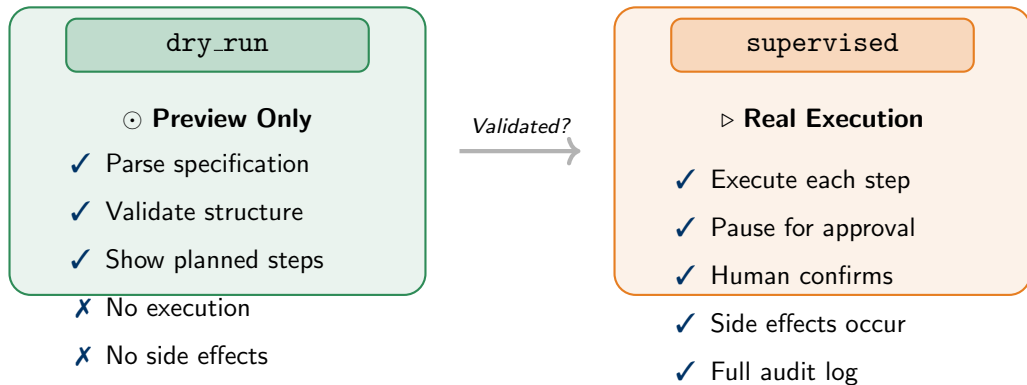
Key Principle

"If it's not in the SDD, it doesn't get coded."

Benefit

Full visibility + Human control + Audit trail

Two Execution Modes



`dry_run`★ The Critical Distinction

is **NOT** sandboxed execution
It is **pure specification parsing**

No code runs. No files created.
No side effects occur.

*"What does this work-
flow **intend** to do?"*

Analogy: `dry_run` is like `terraform plan` — `supervised` is like `terraform apply`

`supervised`

is **REAL** execution
with human oversight

Actions occur on the live system.
Each step requires approval.

"May I proceed with this action?"

Side-Effect Classification

⦿ Read-Only Steps

Auto-execute — No approval needed

- `health_check`
- `validation`
- `data_query`
- `analysis`

These steps only observe; they cannot modify state.

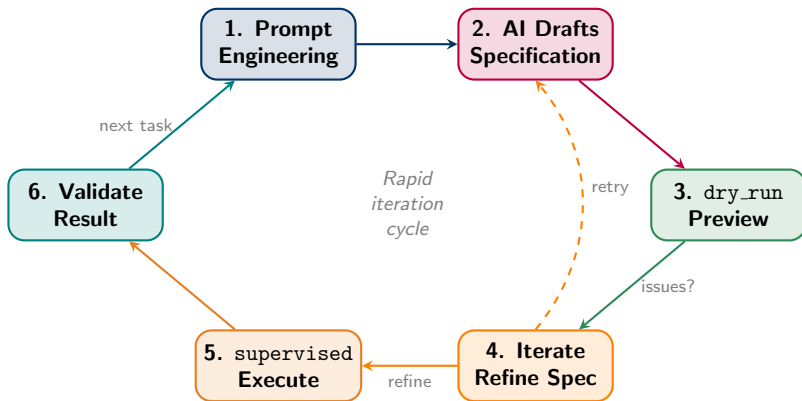
⚠ Side-Effect Steps

Require approval — Human must confirm

- `code_generation`
- `code_modification`
- `command`
- `ingestion`
- `file_delete`
- `git_operation`

In supervised mode: read-only steps auto-execute; side-effect steps pause for approval

The Iterative Development Loop



Loop Phase Details

1. **Prompt Engineering** — Human describes desired outcome in natural language
2. **AI Drafts Spec** — LLM translates intent into structured workflow Markdown
3. **dry_run Preview** — System parses and displays execution plan (no side effects)
4. **Iterate/Refine** — Human reviews plan, adjusts specification if needed
5. **supervised Execution** — Real execution with step-by-step approval gates
6. **Validate Result** — Confirm output, system logs all actions for audit

Why This Works

- **Fast feedback:** `dry_run` is instantaneous — iterate without risk
- **Human in the loop:** Every side effect requires explicit approval

Workflow Specification Format

Step Definition Structure

```
### Step 3: Generate Example Tool
**Type**: code_generation
**Target**: src/tools/ExampleTool.js
**Template**: basic-mcp-tool
**Required**: Yes
**Variables**:
- className: ExampleTool
- toolName: example_operation
- description: Example tool
```

Key Fields

Type Step category (determines approval)

Target File or resource affected

Template Code template to use

Required Can workflow continue if this fails?

Variables Parameters for interpolation

Workflows are just Markdown files — version controlled, diffable, reviewable

Template Interpolation System

Template (basic-mcp-tool.template)

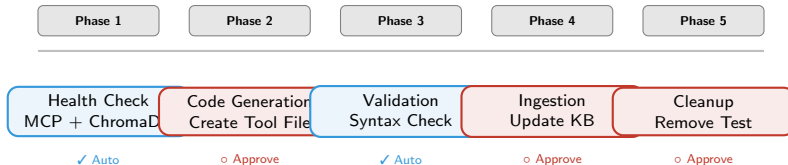
```
class {{className}} {  
  constructor() {  
    this.name = '{{toolName}}';  
    this.description =  
      '{{description}}';  
  }  
  
  async execute(params) {  
    // Implementation  
    return { success: true };  
  }  
}  
  
module.exports = { {{className}} };
```

Generated Output

```
class ExampleTool {  
  constructor() {  
    this.name = 'example_operation';  
    this.description =  
      'Example tool';  
  }  
  
  async execute(params) {  
    // Implementation  
    return { success: true };  
  }  
}  
  
module.exports = { ExampleTool };
```

{{variableName}} → replaced with workflow-specified values

Bootstrap Capability Demo



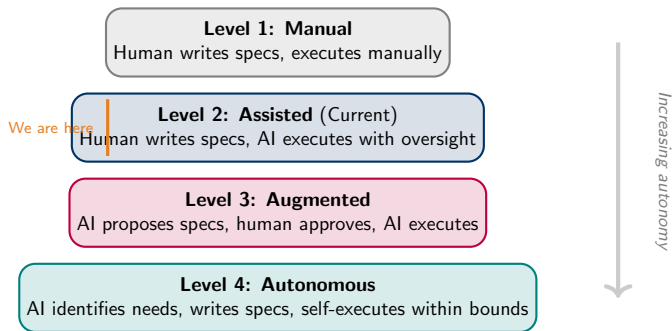
What It Demonstrates

- Full code generation from template
- Syntax validation of generated code
- Knowledge base integration
- Clean teardown

Output

- ExampleBootstrapTool.js — 1,888 bytes
- Valid JavaScript syntax
- MCP tool pattern compliant
- Full execution audit log

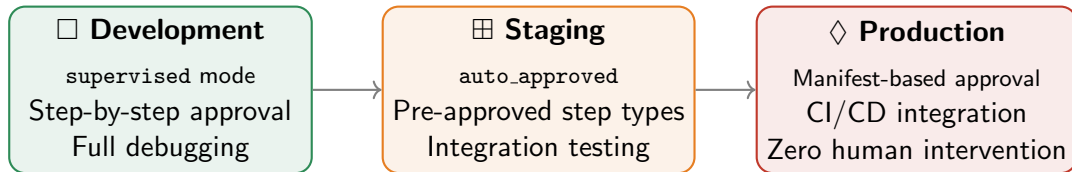
Future: Self-Evolving Systems



Goal: Controlled Self-Evolution

Systems that can bootstrap new capabilities while maintaining human oversight and audit trails

Three Deployment Scenarios



Same workflow specification → Different execution modes → Appropriate oversight level

Summary: Bootstrap Capability

What We Built

- **Spec-Driven Development** — Markdown workflows as source of truth
- **Two-Phase Execution** — `dry_run` + supervised
- **Side-Effect Gates** — Human approval for mutations
- **Template System** — Reusable code generation
- **Full Auditability** — Every action logged

Key Takeaway

Preview before **Execute**

`dry_run` → supervised

Result

AI-assisted code generation with
human oversight and
complete audit trail

Questions?

✉ `Terry.McGuinness@noaa.gov`

NOAA Environmental Modeling Center
Global Workflow MCP Team

Source: Bootstrap Capability White Paper, January 2026

Appendix: Mode Comparison

Characteristic	dry_run	supervised	auto_approved
Parse specification	✓	✓	✓
Execute read-only steps	✗	✓	✓
Execute side-effect steps	✗	✓	✓
Human approval gates	N/A	✓	✗
Files/state modified	✗	✓	✓
Suitable for CI/CD	✗	✗	✓

✓ = Yes ✗ = No N/A = Not applicable